

Chapter 8

Introduction to

JavaScript

INT150 Web Technologies (2/2025)

Sanit Sirisawatvatana

Topics

- What will you learn after finishing this class:
 - Basic Javascript
 - Document Object Model
 - Accessing and Manipulating nodes

JavaScript

- **JavaScript** is a client-side scripting language (runs on the user's machine, not on the server like PHP and Ruby).
- **JavaScript** is reliant on the browser's capabilities and settings.
- **JavaScript** is known as a **dynamic** and **loosely typed** programming language.
- **JavaScript** is a **lightweight** but incredibly powerful scripting language.

What JavaScript can do

- Commonly, [JavaScript](#) is used in a way to add interactivity to a page.
- All of elements, attributes and text on a web page can be accessed by scripts using the [Document Object Model](#) (DOM).
- [Scripts](#) can be written for
 - reacting to user input
 - altering either the contents of page, the CSS styles, or the browser's behavior on the fly

Adding JavaScript to a Page

- Like [CSS](#), a script can be embedded in a document or stored in an external file and linked to the page.
- Both methods use the script element.
- Embedded script

```
<script>
... JavaScript code goes here
</script>
```
- External scripts

```
<script src="my_script.js"></script>
```

Basic JavaScript

- JavaScript is case-sensitive.

myVariable myvariable MyVariable

- A script is made up of a series of statements. Each statement should end with a semicolon (;).

alert("Hello") ;

- // This is a single-line comment
- /* This is a multiple-line comment */

Variable

- Declaring the variable using the `var` keyword.

`var foo = 5 ;` // assign 5 to a variable called foo

- Variable Naming
 - It must start with a letter or an underscore.
 - It may contain letters, digits and underscores
 - It may not contain character space
 - It may not contain special character (! . , / \ + * =).

Data Types

- Undefined

```
var foo ;
```

- Null

```
var foo = null ;
```

- Number

```
var foo = 5 ;
```

- String

```
var foo = "5" ;
```

```
alert(foo+foo) ;
```

- Booleans

```
var foo = true ;
```

<https://www.ajaymatharu.com/javascript-difference-between-undefined-and-null/>

<https://codeburst.io/javascript-null-vs-undefined-20f955215a2>

Array

```
var foo = [ 5 , "Hi", true ] ;
```

```
console.log( foo[0] ) ;
```

```
console.log( foo[1] ) ;
```

```
console.log( foo[2] ) ;
```

```
console.log( foo.length ) ;
```

operators

Comparison operators

==	Is equal to
!=	Is not equal to
===	Is identical to (equal to and of the same data type)
!==	Is not identical to
>	Is greater than
>=	Is greater than or equal to
<	Is less than
<=	Is less than or equal to

Mathematical Operators

+ - * /

+=

++

--

Statements

- if/else statements

```
var num = prompt("Please enter a number") ;  
if (num%2 ==0){  
    alert(num+" is even.") ;  
} else {  
    alert(num+" is odd.") ;  
}
```

- Loop statement

```
var sum = 0 ;  
for(j = 0; j<=10 ; j++ ){  
    sum += j ;  
}  
alert("sum is "+sum) ;
```

- Function

```
function addNumber(a,b){  
    return a+b ;  
}  
  
alert(addNumber(1,2)) ;
```

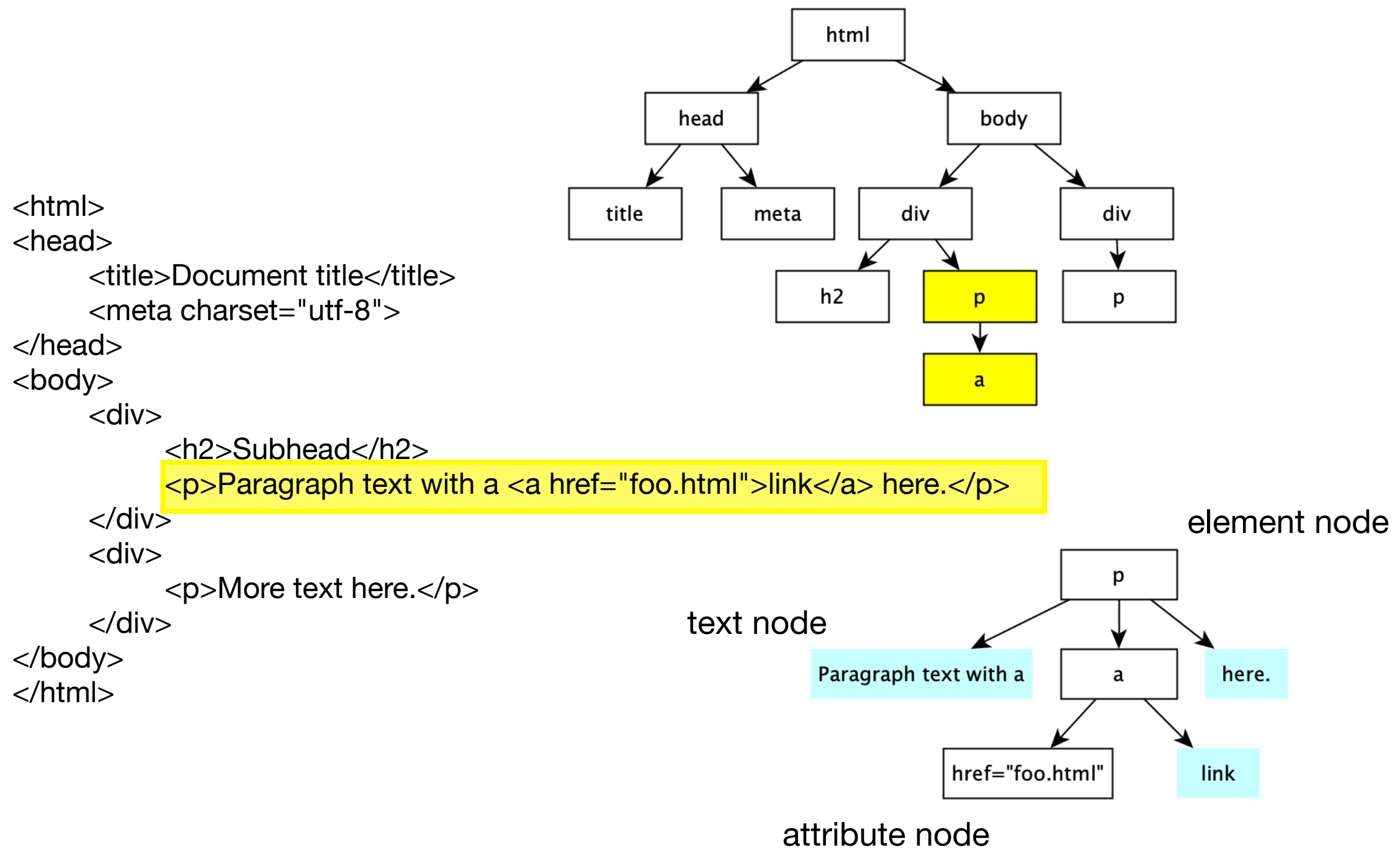
Document Object Model (DOM)

- The DOM gives us a way to access and manipulate the contents of a document.
- The DOM is not only commonly used for HTML, but also for any XML-based language.

Document Object Model (DOM)

- The **DOM** is a programming interface (API) for HTML and XML pages.
- It provides a structured map of the document, as well as a set of methods to interface with the elements contained in the document.
- It translates the markup into a format that **JavaScript** can understand.
- The **DOM** serves as a map to all elements on a page.
- Programmer can use it to find elements by their names or attributes, then add, modify, or delete elements and their content.
- The **DOM** model is constructed as a **tree of objects (nodes)**.
- Nodes are **not only** element nodes, **but also** attribute nodes and text nodes.

Simple document tree



Accessing DOM nodes

- The `document` object in the `DOM` identifies the page itself.
- The `document` object comes with a number of standard properties and methods for accessing collections of elements.
- To access or manipulate a specific part of the page, we use a technique called "`Chaining`."
- This involves linking properties and methods together using `dot notation` (`.`).
- This process creates a logical "`route`" or path through the Document Object Model (DOM) to reach the exact element we need.

Accessing DOM nodes

```
var foo = document.getElementById("beginner").innerHTML
```

1. Look on the page (document)
2. Find the element that has the id value "beginner"
3. Find the HTML content within that element (innerHTML)
4. Save those contents to a variable (foo)

Accessing DOM nodes

```
var foo = document.getElementsByTagName("p")
```

- Look on the page (document), and find the elements by tag name with "p".
- The `getElementsByTagName("p")` returns every paragraph on the page, wrapped in a collection or `nodeList` or arrays in the order they appear in document from top to bottom.
- To access specific paragraphs in the `nodeList`, use their `index`, just like `array`.
- Use `innerHTML` to find the HTML content within that element.

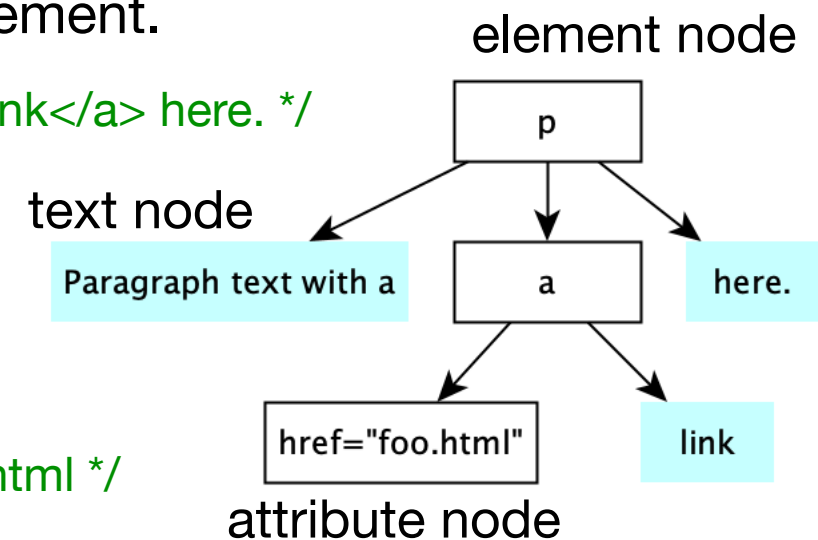
```
foo[0].innerHTML /* Paragraph text with a <a href="foo.html">link</a> here. */
```

```
foo[0].innerText /* Paragraph text with a link here. */
```

```
foo[1].innerHTML /* More text here. */
```

```
foo[0].getElementsByTagName("a")[0].innerHTML /* link */
```

```
foo[0].getElementsByTagName("a")[0].getAttribute("href") /* foo.html */
```



Accessing DOM nodes

- To access each element in the `nodeList` separately, use a `loop` for looping through the array of `nodeList`.

```
var foo = document.getElementsByTagName("p") ;  
for( j=0 ; j < foo.length ; j++) {  
    console.log( foo[ j ].innerHTML ) ;  
}
```

Accessing DOM nodes

- `getElementsByTagName( tagname )`

- returns a collection of an element's child elements with the specified tag name as a `nodeList` object.

```
var paragraphs = document.getElementsByTagName("p") ;
```

- `getElementById( id )`

- returns a single element based on that element's ID (the value of its `id` attribute)

```

```

```
var photo = document.getElementById("lead-photo") ;
```

- `getElementsByClassName( classname )`

- returns every node in the document based on the value of a class attribute.
- Like `getElementsByTagName()`, this returns a `nodeList` that we can reference by index or loop through one at a time.

```
var firstColumn = document.getElementsByClassName("column-a") ;
```

Accessing DOM nodes

- `querySelectorAll( selector )`
 - returns a collection of an element's child elements that match a specified CSS selector(s), as static `nodeList` object.

```
var sidebarPara = document.querySelectorAll(".sidebar p") ;
```

or as complex as selecting an element based on an attribute:

```
var textInput = document.querySelectorAll("input[type='text']") ;
```

- `querySelector( selector )`
 - returns the first child element that matches a specified CSS selector(s) of an element.

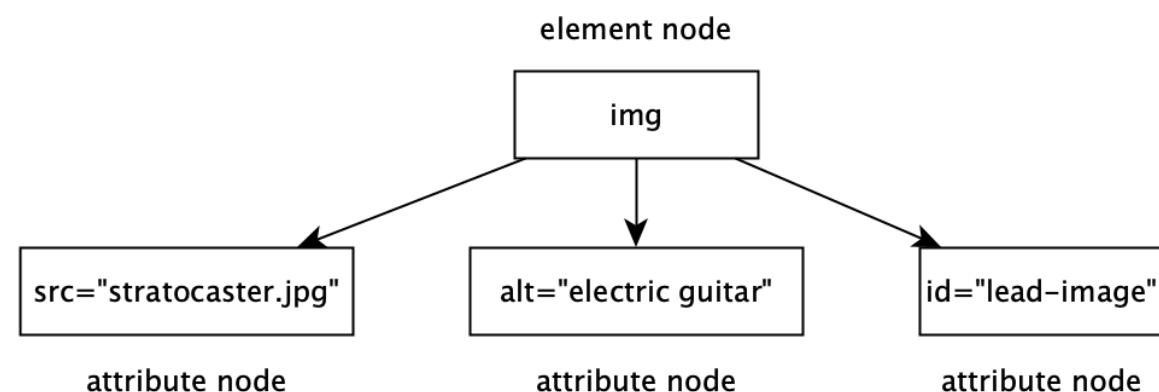
Accessing an attribute value

- `getAttribute( attributename )`
 - returns the value of an attribute with specified name, of an element node.

```

```

```
var bigImage = document.getElementById("lead-image") ;  
alert( bigImage.getAttribute("src") ) ; /* Alerts "stratocaster.jpg" */  
alert( bigImage.getAttribute("alt") ) ; /* Alerts "electric guitar" */
```



Manipulating nodes

- `setAttribute( attributename , value )`
 - adds specified attribute to an element and give it the specified value.
 - If the specified attribute already exists, only the value is set/changed.

```
var bigImage = document.getElementById("lead-image") ;  
bigImage.setAttribute("src", "lespaul.jpg") ;
```

- `innerHTML`
 - is a method for accessing and changing the text and markup inside an element.

```
<p id="greeting">Hello</p>  
var greeting = document.getElementById("greeting") ;  
alert(greeting.innerHTML) ; // return HTML content  
greeting.innerHTML = "New Greeting!" ; // change HTML content
```

Manipulating nodes

- **style**

- This **style** property allows to add, modify, or remove a CSS style from element.
- It works similarly to applying a style with the **inline style** attribute.
- In JavaScript and DOM, property names that are hyphenated in CSS (such as background-color and border-top-width) become camel case (backgroundColor and borderTopWidth).

```
var greeting = document.getElementById("greeting") ;  
greeting.style.color = "#fff" ;  
greeting.style.backgroundColor = "#f58220" ;
```

- It can also be used to get a style value for use elsewhere in the script.

```
var brandColor = document.getElementById("intro").style.backgroundColor ;
```


Adding and removing element

- The **DOM** allows developers to change the document structure itself by adding and removing nodes on the fly.
- `createElement( elementName )`
 - create a new element.
 - This function accepts a single argument: the element to be created.
 - Once we create an element in this way, the new element doesn't appear on the page right away.
 - That new element remains floating in the JavaScript ether until we add it to the document.

```
var newDiv = document.createElement("div")
```


Adding and removing element

- `createTextNode( text )`
 - To enter text into either a newly created element or an existing on the page.
 - Like `createElement( )`, this creates a reference to a new text node that can be stored in a variable and added to the page when required.

```
var ourText = document.createTextNode("This is our text.") ;
```

Adding and removing element

- `appendChild( node )`
 - This method accepts a single argument as the node to add to the DOM.
 - Call the `appendChild` on the existing element that will be its parent in the document structure.

```
var mydiv = document.getElementById("mydiv");  
/* the new element will not display until appending to the document */  
var newpara = document.createElement("p");  
/* create a text node and append to an element */  
var hello = document.createTextNode("Hello, World");  
newpara.appendChild( hello );  
mydiv.appendChild( newpara );
```

Adding and removing element

- `insertBefore( newNode, referenceNode)`
 - inserts a node before a reference node as a child of a specified parent node.
- `replaceChild( newChild, oldChild)`
 - replaces a child node within the given (parent) node.
- `removeChild( child )`
 - removes a child node from the DOM and returns the remove node.

Alternate table row color

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
  </head>
  <body>
    <table>
      <tr>
        <th>Header</th>
      </tr>
      <tr>
        <td>Row 1</td>
      </tr>
      <tr>
        <td>Row 2</td>
      </tr>
      <tr>
        <td>Row 3</td>
      </tr>
    </table>
    <script>
      var foo = document.getElementsByTagName("td") ;
      for ( j=0 ; j < foo.length ; j++ ) {
        if ( j%2 == 0 ) {
          foo[ j ].style.backgroundColor = "cyan" ;
        } else {
          foo[ j ].style.backgroundColor = "yellow" ;
        }
      }
    </script>
  </body>
</html>
```

Header

Row 1

Row 2

Row 3

Demo

<https://stackoverflow.com/questions/3084261/alternate-table-row-color-using-css>

Validate user input

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style type="text/css">
      #errmsg {
        color: red ;
      }
    </style>
  </head>
  <body>
    <label>username: </label><input type="text" id="username" name="username">
    <div id="errmsg"></div>
    <input type="button" value="Submit" onclick="validate()">

    <script>
      function validate(){
        var foo = document.getElementById("errmsg") ;
        var user = document.getElementById("username") ;
        if(user.value==""){
          foo.innerHTML="Please enter username!" ;
        } else {
          foo.innerHTML="" ;
        }
      }
    </script>
  </body>
</html>
```

username:

Submit

username:

Please enter username!

Submit

username: Steve

Submit

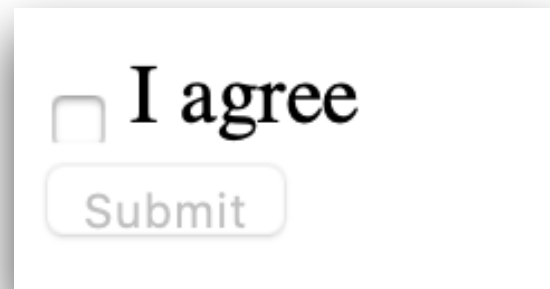
Demo

Enable/Disable Input

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTMLLiveCode</title>
    <style type="text/css">

    </style>
  </head>
  <body>
    <input id ="agree" type="checkbox" onclick="handle()"><label> I agree</label><br>
    <input type="button" id="submit" value="Submit" disabled>

    <script>
      function handle(){
        var foo = document.getElementById("submit") ;
        var agree = document.getElementById("agree") ;
        if ( agree.checked ) {
          foo.removeAttribute("disabled") ;
        }
        else {
          foo.setAttribute("disabled","") ;
        }
      }
    </script>
  </body>
</html>
```



A form with a disabled checkbox and a disabled submit button. The text "I agree" is next to the checkbox, and "Submit" is on the button.



A form with a checked checkbox and an enabled submit button. The text "I agree" is next to the checkbox, and "Submit" is on the button.

Demo

References

- [https://developer.mozilla.org/en-US/docs/Learn/Getting started with the web/JavaScript basics](https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/JavaScript_basics)
- https://www.w3schools.com/js/js_htmlDOM.asp
- [https://developer.mozilla.org/en-US/docs/Web/API/Document Object Model/Introduction](https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction)
- <https://www.guru99.com/practical-code-examples-using-javascript.html>
- <https://stackoverflow.com/questions/7526601/setattributedisabled-false-changes-editable-attribute-to-false>